Secrets Rotation Plan

Item: Amna doc §12 — "Rotation plan for all secrets." **Scope:** Every secret in `projects/physioguidance/secrets/*` (45 total, inventoried 2026-08-17). **Storage:** GCP Secret Manager, region-auto replication, accessed by Cloud Run services via `--set-secrets` flags (per wif-setup-for-amna.md).

1. Rotation policy summary

Class	Cadence	Rotation risk	Downtime
A. Infrastructure credentials (DB, Redis)	On incident, on team departure	Medium	0 (dual-secret pattern)
B. Application-generated (JWT, encryption)	12 months routine + on incident	High	0–5 min (see per-secret notes)
C. Third-party API keys (vendor keys)	12 months routine + on vendor-signaled compromise + on employee departure	Low–Medium	0 (side-by-side deploy)
D. Public config (site keys, DSNs, CA bundles, email addresses)	Never — public by design	None	N/A

Guiding principle: rotate **on trigger** (incident, departure, vendor advisory), not on calendar alone. A 12-month calendar rotation is the ceiling, not the norm. Rotating without cause increases risk (fat-finger deploys) more than it reduces it.

2. Rotation triggers (rotate immediately when any of these fire)

1. **Suspected compromise** — leaked in logs, screenshot, git commit, or public paste.
2. **Team member departure** — anyone with `secretmanager.secretAccessor` on a production secret must be off within 24h and every secret they could read gets rotated within 7 days.
3. **Vendor security advisory** — Moyasar / Tamara / Tabby / Zoom / OpenAI / Resend / Sentry publishes a compromise notice or recommends rotation.
4. **Third-party access-token expiry** — Zoom Server-to-Server OAuth may force credential re-issue on their side.
5. **Migration between environments** — never share prod credentials with staging or dev.
6. **12 months elapsed** since last rotation for Class B (routine ceiling).

3. Secret inventory + per-secret rotation procedure

Class A — Infrastructure credentials

`DATABASE_URL` (Postgres connection incl. password)

- **Rotation flow:**
  1. `gcloud sql users set-password postgres --instance=physioguidance-db-me --password='<new-strong>'` — sets new password on the DB.
  2. `printf '<new-url>' | gcloud secrets versions add DATABASE_URL --data-file=-` — new secret version, old still readable.
  3. Redeploy backend + worker + strapi (`./infra/backend/deploy.sh`, `./infra/worker/deploy.sh`, `./infra/strapi/deploy.sh`) — each service picks up `:latest` on next revision.
  4. Wait 5 min, confirm no 500s on `/health/ready`.
  5. `gcloud secrets versions disable <old-version> --secret=DATABASE_URL`.
- **Zero-downtime trick:** Postgres allows only one password per user. If dual-password is required, temporarily create a second Postgres user, switch backend, retire old user. For a routine rotation, a ~2-second connection-drop is acceptable — Cloud Run rolls services one revision at a time.
- **Recovery test:** run this in the DR drill (§ 7).

`STRAPI_DATABASE_URL` + `STRAPI_DATABASE_PASSWORD`

- Same flow as `DATABASE_URL` but for the Strapi user on the same Postgres instance.
- Rotate at the same cadence as `DATABASE_URL`.

`REDIS_HOST`

- Not a credential — it's a hostname. Memorystore Redis in BASIC tier has no password (VPC-internal). No rotation needed.
- **Only rotate if** the Redis instance is recreated (migration, tier change) — update the secret with the new hostname and redeploy consumers.

`REDIS_CA_CERT`

- Not a secret — public CA bundle. No rotation unless GCP rotates their Memorystore CA (extremely rare; would be a project-wide GCP change).

Class B — Application-generated secrets

`JWT_SECRET` (short-lived access tokens)

- **Rotation flow:**

1. Generate: `openssl rand -base64 64 | tr -d '\n'`.
2. `printf '<new>' | gcloud secrets versions add JWT_SECRET --data-file=-`.
3. Redeploy backend + worker.
4. Cloud Run rolling deploy takes ~2 min. Existing access tokens signed by the old key remain valid until they expire (default 15 min).
5. Disable old version 1 hour after deploy (any active session had time to refresh).

- **User impact:** users get an `EXPIRED_ACCESS` on their next API call after their old token expires; the frontend auto-refreshes via `/auth/refresh`. Transparent.
- **Rotation cadence:** 12 months + on-incident.

`JWT_REFRESH_SECRET` (long-lived refresh tokens)

- **Rotation flow:**
  1. Generate + add version + redeploy (same as JWT_SECRET).
  2. All existing refresh tokens are invalidated. Every user is logged out and must re-login.
- **User impact:** universal forced re-login. Communicate 24h in advance via in-app banner + email.
- **Rotation cadence:** 12 months routine + on-incident. Coordinate with a low-traffic window (Friday evening SA time).
- **On incident (compromised refresh tokens):** rotate immediately — accept the forced logout as the cost of containment.

`PII_ENCRYPTION_KEY` (AES-256-GCM key for `encryptPII` / `decryptPII`)

- **Highest-risk rotation.** All PII rows in the DB are encrypted with the CURRENT key. Rotating without a re-encryption migration will make old rows unreadable.
- **Rotation flow (must ship a migration script — do NOT rotate this key alone):**
  1. Add a `PII_ENCRYPTION_KEY_OLD` secret with the current value.
  2. Generate new key, deploy as `PII_ENCRYPTION_KEY`.
  3. Update `encryptPII` / `decryptPII` code to accept a version prefix (`v2:...` for new, `v1:...` for old) and decrypt accordingly. Encrypt with new key going forward.
  4. Ship a background job (Cloud Run Job) that re-encrypts every PII row: read → decrypt with old → encrypt with new → write. Chunk in batches of 500.
  5. When migration reaches 100%, remove `PII_ENCRYPTION_KEY_OLD` and drop the version-prefix branch.
- **Rotation cadence:** Only on suspected compromise. The migration cost (dev + backend downtime risk + audit-log noise) exceeds the routine security benefit at current data volume. Revisit at 100k+ users.
- **PDPL notes:** rotation is documented as an accepted compensating control (see pdpl-controls-matrix.csv) — Article 13 requires reasonable technical safeguards, not scheduled rotation.

`RECONCILIATION_CRON_TOKEN`

- Internal token used by the payment-reconciliation cron controller to authenticate with itself.
- **Rotation flow:** generate + add version + redeploy. Rotation is transparent (single-service token).
- **Rotation cadence:** 12 months + on-incident.

Strapi secrets — `STRAPI_ADMIN_JWT_SECRET`, `STRAPI_API_TOKEN_SALT`, `STRAPI_APP_KEYS`, `STRAPI_JWT_SECRET`, `STRAPI_TRANSFER_TOKEN_SALT`

- Same shape as JWT_SECRET / JWT_REFRESH_SECRET but for Strapi.
- **Rotation flow:** generate new value with `openssl rand -base64 32` (Strapi expects base64), add version, redeploy Strapi.
- **User impact:** rotating `STRAPI_ADMIN_JWT_SECRET` or `STRAPI_JWT_SECRET` logs out all Strapi admins (Amna + team). Communicate first.
- `STRAPI_APP_KEYS` **special note:** this is a comma-separated list. Rotation supports a **two-key window** — put both old+new in the list, deploy, wait for sessions to naturally expire, then remove old. Zero downtime for content editors.
- **Rotation cadence:** 12 months + on Strapi security advisory.

Class C — Third-party API keys

Moyasar — `MOYASAR_PUBLISHABLE_KEY_{LIVE,TEST}`, `MOYASAR_SECRET_KEY_{LIVE,TEST}`, `MOYASAR_WEBHOOK_SECRET_TEST`

- **Rotation flow:**
  1. Log in to Moyasar dashboard.
  2. Generate new keys (Moyasar supports side-by-side old+new for a grace window).
  3. Add new value as secret version, redeploy backend.
  4. In Moyasar dashboard, revoke old keys.
- **Webhook secret rotation:** Moyasar signs webhook payloads with the secret. Backend must accept both old+new secrets for the transition window (~5 min); implement `validateWebhookSignature` to try new first, fall back to old. Discard old after 24h.
- **Rotation cadence:** 12 months + on Moyasar advisory + on team departure with dashboard access.
- **Note:** Missing `MOYASAR_WEBHOOK_SECRET_LIVE` — verify live-mode webhook signature validation is wired before revenue traffic scales up.

Tabby — `TABBY_SECRET_KEY`, `TABBY_WEBHOOK_SECRET`

- Same pattern as Moyasar (dashboard → generate → add secret version → deploy → revoke old).
- Webhook signature validation supports the two-secret window.

Tamara — `TAMARA_API_TOKEN`, `TAMARA_NOTIFICATION_TOKEN`, `TAMARA_PUBLIC_KEY`

- API/notification tokens: dashboard-driven rotation. Add new version, deploy, revoke old.

- Public key: rotates when Tamara rotates their signing key — they'll notify. Update the secret, redeploy.

Zoom — `ZOOM_ACCOUNT_ID`, `ZOOM_CLIENT_ID`, `ZOOM_CLIENT_SECRET`, `ZOOM_WEBHOOK_SECRET_TOKEN`

- **Server-to-Server OAuth app.** Rotation flow:
  1. In Zoom Marketplace → your S2S OAuth app → Regenerate client secret.
  2. Zoom returns new secret ONCE — copy immediately.
  3. Add secret version + redeploy backend.
  4. Old secret is invalid the moment you regenerate — this is NOT dual-secret. Do the rotation with the deploy queued and ready to fire.
- **Webhook secret:** rotate independently via the "Feature" tab of the app. Same one-shot pattern.
- **Rotation cadence:** 12 months + on team departure + on Zoom advisory.
- **Downtime:** Zoom API calls will 401 for the ~2 min between regenerating and the new Cloud Run revision being live. Zoom SDK retries; short-lived failures do not create bad DB state.

OpenAI — `OPENAI_API_KEY`

- **Rotation flow:**
  1. OpenAI dashboard → API keys → Create new key.
  2. Add secret version, redeploy any consumer service.
  3. Delete old key in OpenAI dashboard.
- **Usage today:** transcript summarization + optional Q&A features. Non-critical — a failed OpenAI call degrades gracefully.
- **Rotation cadence:** 12 months + on departure of anyone with OpenAI console access.

Resend / SMTP — `RESEND_API_KEY`, `SMTP_{HOST,PORT,USER,PASS}`

- Resend key: dashboard rotation → add version → deploy.
- SMTP creds: only used as fallback when Resend is misconfigured. Rotate on departure of anyone with SMTP-provider console access.
- Impact of stale key: outbound email fails. `SendGrid` (via SMTP fallback) or a redeploy fixes.

NELC — `NELC_LRS_PASSWORD`, `NELC_LRS_PASSWORD_PROD`

- LRS = Learning Record Store credentials. Provided by NELC integration desk.
- **Rotation:** contact NELC, request new credentials, add version, redeploy `xapi` module consumers.
- **Rotation cadence:** on NELC-signaled change. Not routine.

reCAPTCHA — `RECAPTCHA_SECRET_KEY`, `NEXT_PUBLIC_RECAPTCHA_SITE_KEY`

- `RECAPTCHA_SECRET_KEY` is the backend-verified secret. Rotate on Google reCAPTCHA console:
  1. Create new site key + secret pair.

2. Update both secrets (public + private) simultaneously.
3. Deploy frontend + backend together (site key is baked into the frontend bundle).
4. Delete old key pair.

- **Note:** `NEXT_PUBLIC_RECAPTCHA_SITE_KEY` is publicly served to browsers — it's not confidential. Rotating still requires a coordinated frontend redeploy so the browser gets the new site key.

Sentry — `SENTRY_API_TOKEN`, `SENTRY_AUTH_TOKEN`, `SENTRY_WEBHOOK_SECRET`, `SENTRY_DSN_BACKEND`, `NEXT_PUBLIC_SENTRY_DSN`

- DSNs (`SENTRY_DSN_BACKEND`, `NEXT_PUBLIC_SENTRY_DSN`) are NOT secrets — they identify the project + are safe to expose. No rotation unless project migration.
- Tokens (`SENTRY_API_TOKEN` used at runtime, `SENTRY_AUTH_TOKEN` used at CI build): rotate 12 months + on departure of anyone with Sentry access.
- Webhook secret: rotate on Sentry-webhook-integration reconfigure.

Class D — Public config (no rotation required)

Secret	Why it's not a secret
`MAIL_FROM`	Email address
`MAIL_REPLY_TO`	Email address
`NEXT_PUBLIC_RECAPTCHA_SITE_KEY`	Bundled into browser JS
`NEXT_PUBLIC_SENTRY_DSN`	Bundled into browser JS
`SENTRY_DSN_BACKEND`	DSN — public per Sentry docs
`REDIS_CA_CERT`	Public CA bundle
`REDIS_HOST`	VPC-internal hostname
`MOYASAR_PUBLISHABLE_KEY_{LIVE,TEST}`	"Publishable" — designed for client exposure
`TAMARA_PUBLIC_KEY`	Public key used for signature verification

These live in Secret Manager for **operational convenience** (single source of truth, no `.env` drift). Rotate only on environment migration.

---

4. Zero-downtime dual-secret pattern

For secrets where an abrupt cutover would break in-flight requests, use the **dual-secret window**:

1. Create secret `<NAME>_NEW` alongside existing `<NAME>`.

2. Deploy code that reads both and accepts either (for verification: try new first, fall back to old).
3. Cut writers over to new.
4. Wait for the old secret's usage TTL to expire (webhook age, session age, etc.).
5. Remove `<NAME>_NEW`, promote its value into `<NAME>`.

Applies to: webhook signature secrets (Moyasar, Tabby, Tamara, Zoom, Sentry), JWT_SECRET, `STRAPI_APP_KEYS`. **Does NOT apply to:** JWT_REFRESH_SECRET (design: rotating this SHOULD invalidate old sessions), Postgres passwords (Postgres allows only one).

---

5. Rotation runbook (copy-paste template)

```
SECRET=<name>
NEW_VALUE="$(openssl rand -base64 64 | tr -d '\n')"  # or paste vendor value

# 1. Add new version
printf '%s' "$NEW_VALUE" | gcloud secrets versions add "$SECRET" --data-file=-

# 2. Confirm consumers by checking which services mount this secret
gcloud run services list --region=me-central2 --format="value(metadata.name)" | \
  while read svc; do
    if gcloud run services describe "$svc" --region=me-central2 \
        --format="value(spec.template.spec.containers[0].env)" 2>/dev/null | \
        grep -q "$SECRET"; then
      echo "→ mounted on $svc"
    fi
  done

# 3. Redeploy consumers via infra scripts
./infra/backend/deploy.sh   # if api service
./infra/worker/deploy.sh    # if worker
./infra/strapi/deploy.sh    # if strapi

# 4. Verify no 500s on /health/ready
curl -s https://api.physioguidance.org/health/ready | jq .

# 5. After confidence window (1h for JWT, 24h for webhooks, immediate for DB pw)
OLD_VERSION=$(gcloud secrets versions list "$SECRET" --format="value(name)" | sed -n '2p')
gcloud secrets versions disable "$OLD_VERSION" --secret="$SECRET"

# 6. Update the rotation log:
echo "$(date -u +%Y-%m-%dT%H:%M:%SZ) rotated $SECRET" >> \
  ~/Desktop/PhysioGuidance-Handover/secrets-rotation-log.md
```

---

6. Access audit — who can read each secret today

Access is granted via IAM binding on the secret (`roles/secretmanager.secretAccessor`) or at project level. As of 2026-08-17:

- **Runtime service accounts** — `api-runtime@` , `worker-runtime@` , `web-runtime@` , `strapi-runtime@` , `malware-runtime@` each have per-secret accessor grants for only the secrets they need. See least-privilege-service-accounts-for-amna.sh.
- **Human owners** — Amna (`amna@physioguidance.org`) has project-level `roles/owner` .
- **Handover holdouts** — Ghulam (`ghulam@physioguidance.org`) currently retains `roles/owner` for handover completion. **Removal is Amna's step**, per alert-recipients-status.md.

On any human departure, all Class A + Class C secrets that person could read must rotate within 7 days.

---

7. DR + rotation drill (recommended quarterly)

Rotation procedures are **not rehearsed** as of handover. Real rotations always take longer than the doc suggests. Suggested quarterly drill (30 min):

1. Rotate `RECONCILIATION_CRON_TOKEN` — lowest-blast-radius internal token. Verify cron still fires next tick.
2. Rotate `RESEND_API_KEY` — verify a test email delivers.
3. Read + record actual time-to-complete + any snags in the rotation log.
4. Once/year: rotate `JWT_SECRET` in low-traffic window as the full drill.

First drill: within 90 days of handover complete.

---

8. Missing pieces (for Amna to file)

- **`MOYASAR_WEBHOOK_SECRET_LIVE`** — production webhook secret not yet configured. Required before live Moyasar traffic. Amna's step: obtain from Moyasar live account + add to Secret Manager + redeploy backend.
- **Formal rotation log** — start `secrets-rotation-log.md` in the handover folder; append one line per rotation. Auditors will ask for this.
- **Vendor-notification subscriptions** — subscribe `amna@physioguidance.org` to security advisories for Moyasar, Tabby, Tamara, Zoom, OpenAI, Resend, Sentry. Most have a "security" or "changelog"

mailing list.

---

9. Summary for Amna

- Every production secret has a documented rotation path.
- Rotate on trigger (incident, departure, vendor advisory), not on calendar alone.
- 12-month ceiling for routine Class B secrets; Class D never rotates.
- Two secrets need dual-key handling: `STRAPI_APP_KEYS` (built-in support), any webhook secret (implement two-secret verification for zero-downtime rotation).
- Only `PII_ENCRYPTION_KEY` is fundamentally hard to rotate — it needs a migration job, don't rotate solo.
- No urgent action today. Highest-value next steps: file `MOYASAR_WEBHOOK_SECRET_LIVE`, start the rotation log, schedule the first quarterly drill.